

Assignment 1 (Coding)

● Graded

Student

Jose David Ferris Gomez

Total Points

20 / 20 pts

Autograder Score

20.0 / 20.0

Passed Tests

1a-0-basic) Basic test case. (1/1)

1a-1-hidden) Test multiple instances of the same word in a sentence. (1/1)

1b-0-basic) Basic sanity check for learning correct weights on two training and testing examples each. (1/1)

1b-1-basic) Test correct overriding of positive weight due to one negative instance with repeated words. (2/2)

1b-2-basic) Test classifier on real polarity dev dataset. (2/2)

1c-0-basic) test correct generation of random dataset labels (2/2)

1c-1-basic) test that the randomly generated example actually coincides with the given weights (2/2)

1d-0-basic) test basic character n-gram features (2/2)

1d-1-hidden) test feature extraction on repeated character n-grams (2/2)

2b-0-basic) test basic k-means on hardcoded datapoints. (1/1)

2b-1-hidden) test stability of cluster assignments. (1/1)

2b-2-hidden) test stability of cluster locations. (1/1)

2b-3-hidden) scaling kmean. (2/2)

Autograder Results

1a-0-basic) Basic test case. (1/1)

1a-1-hidden) Test multiple instances of the same word in a sentence. (1/1)

1b-0-basic) Basic sanity check for learning correct weights on two training and testing examples each. (1/1)

1b-1-basic) Test correct overriding of positive weight due to one negative instance with repeated words. (2/2)

1b-2-basic) Test classifier on real polarity dev dataset. (2/2)

1c-0-basic) test correct generation of random dataset labels (2/2)

1c-1-basic) test that the randomly generated example actually coincides with the given weights (2/2)

1d-0-basic) test basic character n-gram features (2/2)

1d-1-hidden) test feature extraction on repeated character n-grams (2/2)

2b-0-basic) test basic k-means on hardcoded datapoints. (1/1)

2b-1-hidden) test stability of cluster assignments. (1/1)

2b-2-hidden) test stability of cluster locations. (1/1)

2b-3-hidden) scaling kmean. (2/2)

Submitted Files

```

1  #!/usr/bin/python
2
3  import random
4  from collections import defaultdict
5  from typing import Callable, Dict, List, Tuple, TypeVar, DefaultDict
6  from util import *
7
8  FeatureVector = Dict[str, int]
9  WeightVector = Dict[str, float]
10 Example = Tuple[FeatureVector, int]
11
12 #####
13 # Problem 1: binary classification
14 #####
15
16 #####
17 # Problem 1a: feature extraction
18
19
20 def extractWordFeatures(x: str) -> FeatureVector:
21     """
22     Extract word features for a string x. Words are delimited by
23     whitespace characters only.
24     @param string x:
25     @return dict: feature vector representation of x.
26     Example: "I am what I am" --> {'I': 2, 'am': 2, 'what': 1}
27     """
28     pass
29     # ### START CODE HERE ###
30
31     # 1.Creation of the empty feature vector 'review_vector'
32     # 2.Split the input string at empty spaces
33     # 3.Iterate the dictionary, checking if the word is there
34     # If not, the word is introduced and the value is updated.
35     # If it is, only the value is updated (done with dict.get function)
36
37     review_phi = {}
38     words = x.split()
39
40     for w in words:
41         review_phi[w] = review_phi.get(w, 0) + 1
42
43     return (review_phi)
44
45     # ### END CODE HERE ###
46
47
48 #####
49 # Problem 1b: stochastic gradient descent

```

```

50
51 T = TypeVar("T")
52
53
54 def learnPredictor(
55     trainExamples: List[Tuple[T, int]],
56     validationExamples: List[Tuple[T, int]],
57     featureExtractor: Callable[[T], FeatureVector],
58     numEpochs: int,
59     eta: float,
60 ) -> WeightVector:
61     """
62     Given |trainExamples| and |validationExamples| (each one is a list of (x,y)
63     pairs), a |featureExtractor| to apply to x, and the number of epochs to
64     train |numEpochs|, the step size |eta|, return the weight vector (sparse
65     feature vector) learned.
66
67     You should implement stochastic gradient descent.
68
69     Notes:
70     - Only use the trainExamples for training!
71     - You should call evaluatePredictor() on both trainExamples and validationExamples
72     to see how you're doing as you learn after each epoch.
73     - The predictor should output +1 if the score is precisely 0.
74     """
75     weights = {} # feature => weight
76     # #### START CODE HERE ###
77
78     # If the Score (phi(x) . w) is exactly 0, output +1. Otherwise, output -1.
79     def predictor(x: T) -> int:
80         return 1 if dotProduct(featureExtractor(x), weights) >= 0 else -1
81     # Stochastic Gradient Descent:
82     # First, iterate over the number of epochs.(1....T or numEpochs)
83     # Then, iterate over each training example (x,y).
84     # After phi, the margin is computed.
85     for epoch in range(numEpochs):
86         for x, y in trainExamples:
87             phi = featureExtractor(x)
88             margin = dotProduct(weights, phi) * y
89             # If margin is less than 1, loss hinge is greater than 0, thereby not converged. So, update
90             # weights.
91             if margin < 1:
92                 # Update weights according to the formula.
93                 increment(weights, eta * y, phi)
94
95             # Monitor training progress.
96             trainError = evaluatePredictor(trainExamples, predictor)
97             validationError = evaluatePredictor(validationExamples, predictor)
98             print(f"Epoch {epoch}: train error = {trainError}, validation error = {validationError}")
99
100     # #### END CODE HERE ###
101     return weights

```

```

101
102
103 #####
104 # Problem 1c: generate test case
105
106
107 def generateDataset(numExamples: int, weights: WeightVector) -> List[Example]:
108     """
109     Return a set of examples (phi(x), y) randomly which are classified correctly by
110     |weights|.
111     """
112     random.seed(42)
113
114     # Return a single example (phi(x), y).
115     # phi(x) should be a dict whose keys are a subset of the keys in weights
116     # and values can be anything (randomize!) with a score for the given weight vector.
117     # y should be 1 or -1 as classified by the weight vector.
118     # y should be 1 if the score is precisely 0.
119
120     # Note that the weight vector can be arbitrary during testing.
121     def generateExample() -> Tuple[Dict[str, int], int]:
122         phi = None
123         y = None
124         # ### START CODE HERE ###
125         phi = {} #Empty feature vector as a dictionary
126         keys = list(weights.keys())
127
128         if keys:
129             # choose a non-empty subset of features randomly
130             subset_size = random.randint(1, len(keys))
131             chosen_keys = random.sample(keys, subset_size)
132             for k in chosen_keys:
133                 # random non-zero value to allow positive/negative scores
134                 phi[k] = random.choice([-5, -4, -3, -2, -1, 1, 2, 3, 4, 5])
135
136             score = dotProduct(phi, weights)
137
138             """
139             #-----SCORE DEBUGGING-----
140             #Even though the phi and weights are non-zero, some errors
141             # may lead to zero score. This block is to debug such cases.
142             print("Score =", score)
143             # avoid zero score so label is well-defined
144             #check_zero_weights = 0 in weights.values()
145             #print(check_zero_weights)
146             if score == 0:
147                 print("#####SCORE IS ZERO#####")
148                 k0 = chosen_keys[0]
149                 phi[k0] += 1 if weights[k0] >= 0 else -1
150                 score = dotProduct(phi, weights)
151             #-----
152             """

```

```

153
154     y = 1 if score >= 0 else -1
155 else:
156     # no weights: return empty features with positive label
157     phi = {}
158     y = 1
159     # ### END CODE HERE ###
160     return (phi, y)
161
162 return [generateExample() for _ in range(numExamples)]
163
164
165 #####
166 # Problem 1d: character features
167
168
169 def extractCharacterFeatures(n: int) -> Callable[[str], FeatureVector]:
170     """
171     Return a function that takes a string |x| and returns a sparse feature
172     vector consisting of all n-grams of |x| without spaces mapped to their n-gram counts.
173     EXAMPLE: (n = 3) "I like tacos" --> {'lli': 1, 'lik': 1, 'ike': 1, ...
174     You may assume that n >= 1.
175     """
176
177     def extract(x):
178         # ### START CODE HERE ###
179         phi = {}
180         s = x.replace(" ", "") # Remove spaces from the string
181         # Iterate over the string to extract n-grams avoiding index errors
182         #The 'gram' variable stores the n-gram substring on phi update.
183         for i in range(len(s) - n + 1):
184             gram = s[i : i + n]
185             phi[gram] = phi.get(gram, 0) + 1
186         return phi
187         # ### END CODE HERE ###
188
189     return extract
190
191
192 #####
193 # Problem 1e:
194 #
195 # Helper function to test 1e.
196 #
197 # To run this function, run the command from terminal with `n` replaced
198 #
199 # $ python -c "from submission import *; testValuesOfN(n)"
200 #
201
202
203 def testValuesOfN(n: int):
204     """

```

```

205 Use this code to test different values of n for extractCharacterFeatures
206 This code is exclusively for testing.
207 Your full written solution for this problem must be submitted.
208 """
209 trainExamples = readExamples("polarity.train")
210 validationExamples = readExamples("polarity.dev")
211 featureExtractor = extractCharacterFeatures(n)
212 weights = learnPredictor(
213     trainExamples, validationExamples, featureExtractor, numEpochs=20, eta=0.01
214 )
215 outputWeights(weights, "weights")
216 outputErrorAnalysis(
217     validationExamples, featureExtractor, weights, "error-analysis"
218 ) # Use this to debug
219 trainError = evaluatePredictor(
220     trainExamples,
221     lambda x: (1 if dotProduct(featureExtractor(x), weights) >= 0 else -1),
222 )
223 validationError = evaluatePredictor(
224     validationExamples,
225     lambda x: (1 if dotProduct(featureExtractor(x), weights) >= 0 else -1),
226 )
227 print(
228     (
229         "Official: train error = %s, validation error = %s"
230         % (trainError, validationError)
231     )
232 )
233
234
235
236 #####
237 # Problem 2b: K-means
238 #####
239
240
241 def kmeans(
242     examples: List[Dict[str, float]], K: int, maxEpochs: int
243 ) -> Tuple[List, List, float]:
244     """
245     examples: list of examples, each example is a string-to-float dict representing a sparse vector.
246     K: number of desired clusters. Assume that 0 < K <= |examples|.
247     maxEpochs: maximum number of epochs to run (you should terminate early if the algorithm
248     converges).
249     Return: (length K list of cluster centroids,
250             list of assignments (i.e. if examples[i] belongs to centers[j], then assignments[i] = j),
251             final reconstruction loss)
252     """
253     # ### START CODE HERE ###
254     #-----High level explanation from the slides:-----
255     # K-means clustering: assign points to nearest center and update centers.
256     # Centers are initialized by sampling K examples.

```

```

256 # Distances are computed using expanded squared norm for efficiency, researched as:
257 #  $\|x-c\|^2 = \|x\|^2 - 2\langle x, c \rangle + \|c\|^2$ 
258 # Handles empty clusters by reassigning a random example as its center. To avoid division by zero.
259
260 # Set random seed for starting centers
261 random.seed(42)
262 n = len(examples)
263 #Select K random examples as initial centers
264 centers = random.sample(examples, K)
265 # Precompute example norms for efficiency and store initial assignments
266 exampleNorms = [dotProduct(x, x) for x in examples]
267 assignments = [-1] * n
268
269 ## -----Main loop:-----
270 # 1) Assignment step: assign each x_i to nearest center.
271 # 2) Check for convergence (assignments unchanged).
272 # 3) Update step: recompute centers as average of points in each cluster
273 for _ in range(maxEpochs):
274     centerNorms = [dotProduct(c, c) for c in centers]
275     newAssignments = []
276     totalCost = 0.0
277
278     # Assignment step
279     for i, x in enumerate(examples):
280         bestCluster = None
281         bestCost = None
282         for j, c in enumerate(centers):
283             cost = exampleNorms[i] - 2 * dotProduct(x, c) + centerNorms[j]
284             if bestCost is None or cost < bestCost:
285                 bestCost = cost
286                 bestCluster = j
287             newAssignments.append(bestCluster)
288             totalCost += bestCost
289
290     # Stop if converged
291     if newAssignments == assignments:
292         assignments = newAssignments
293         break
294     assignments = newAssignments
295
296     # Update centers
297     clusterSums = defaultdict(float)
298     counts = [0] * K
299     for x, z in zip(examples, assignments):
300         counts[z] += 1
301         for f, v in x.items():
302             clusterSums[z][f] += v
303
304     newCenters = []
305     for j in range(K):
306         #if a cluster is empty, reassign a random example as its center
307         if counts[j] == 0:

```



```
308         newCenters.append(random.choice(examples))
309         continue
310     inv = 1.0 / counts[j]
311     center = {}
312     for f, v in clusterSums[j].items():
313         val = v * inv
314         if val != 0:
315             center[f] = val
316     newCenters.append(center)
317     centers = newCenters
318
319 # Final loss
320 centerNorms = [dotProduct(c, c) for c in centers]
321 totalCost = 0.0
322 # Compute final reconstruction loss once converged
323 for i, x in enumerate(examples):
324     j = assignments[i]
325     c = centers[j]
326     totalCost += exampleNorms[i] - 2 * dotProduct(x, c) + centerNorms[j]
327
328 return centers, assignments, totalCost
329 # ### END CODE HERE ###
330
```
